# `scripts/run-release-canary-containerized.sh` — User Guide

**Last verified:** 2026-08-12 (initial version — closes the release-verification gap on hosts without Genymotion) **Inheritance:** HelixConstitution §11.4.18 + Lava §6.Z (Anti-Bluff Distribute Guard) + §6.X (Container-Submodule Emulator Wiring) + §6.AH (Container/VM-only emulators)

## Overview

Runs a real, on-device cold-start canary against a **release-signed** APK — the exact artifact about to be distributed, not a debug stand-in. Sibling to `scripts/run-release-canary.sh` (LVA-077), which uses a Genymotion VM. This script uses the Containers submodule's `cmd/emulator-canary` binary through its containerized (podman/docker) runner instead, for gate hosts where Genymotion is not installed.

## Why this exists

Two real, confirmed constraints make instrumented (JUnit) Challenge Tests unusable against a release build:

1. AGP does not generate a matching `connectedReleaseAndroidTest` task without explicit `testBuildType` configuration (not present in this project).
2. Even if forced, `isDebuggable=true` **disables all R8 optimization/obfuscation regardless of `isMinifyEnabled`** (confirmed live: Gradle emits `WARNING: BuildType 'release' is both debuggable and has 'isMinifyEnabled' set to true. All code optimizations and obfuscation are disabled for debuggable builds.`). Testing a debuggable release build proves nothing about the R8-processed code that actually ships — exactly the class of bug behind the 1.2.19 incident that birthed §6.AA.

The Containers submodule's `RunCanary` (in `pkg/emulator/canary.go`) sidesteps both: it installs the real release APK, launches it via `adb shell am start`, and watches `logcat` for FATAL/AndroidRuntime crash signatures — no instrumentation required. Until this script's underlying Go changes landed, `RunCanary` only supported the host-direct runner, which violates this project's own §6.AH ("virtual

devices MUST run in containers or VMs, no exception") on Linux. The Containers submodule now exposes `--runner=auto|host-direct|containerized` on `emulator-canary`, matching `emulator-matrix`'s existing convention.

## Usage

```
./scripts/run-release-canary-containerized.sh \
  --apk releases/1.3.15/android-release/
        digital.vasic.lava.client-1.3.15-release.apk \
  --package digital.vasic.lava.client
```

Optional flags: `--activity` (default `.MainActivity`), `--avd` (default `CZ_API34_Phone:34:phone`), `--evidence-dir`, `--container-image`, `--container-runtime`, `--no-build`.

## Exit codes

- `0` — activity reached RESUMED state AND no FATAL detected in logcat (canary PASS)
- `1` — activity did not resume OR a FATAL/AndroidRuntime crash was detected (canary FAIL)
- `2` — configuration error (missing APK, missing required flag, boot failure)

## Evidence

Writes `<evidence-dir>/canary-attestation.json` (structured result: `activity_resumed`, `fatal_detected`, `passed`, boot/timing) and `<evidence-dir>/logcat.txt` (the full captured logcat window). Per §6.J anti-bluff posture, the primary assertion is `activity_resumed=true AND fatal_detected=false` — "installed without error" alone is never a PASS.

## Choosing between the two release-canary scripts

- `scripts/run-release-canary.sh` — use when a Genymotion VM is running (`gmtool` installed, VM booted). Faster iteration once a VM is already up.
- `scripts/run-release-canary-containerized.sh` (this script) — use when Genymotion is unavailable. Cold-boots a fresh containerized KVM emulator per run (slower per-invocation, but works on any Linux host with podman/docker + `/dev/kvm`, no separate VM product required).

Both produce comparable pass/fail semantics; neither is a substitute for real operator on-device confirmation per §6.AA clause 2 for tag-gating releases — they close the *automated* verification gap, not the human sign-off requirement.